

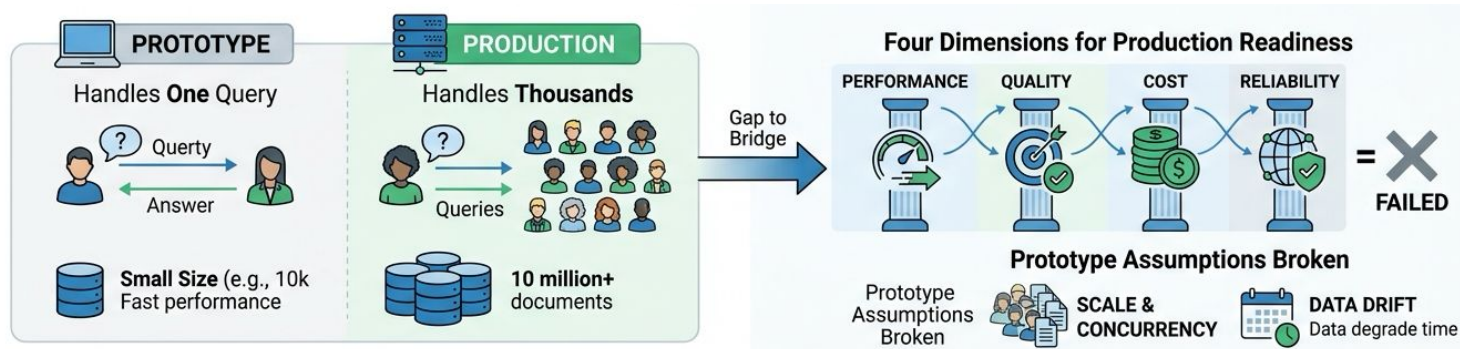


U.S. General Services
Administration

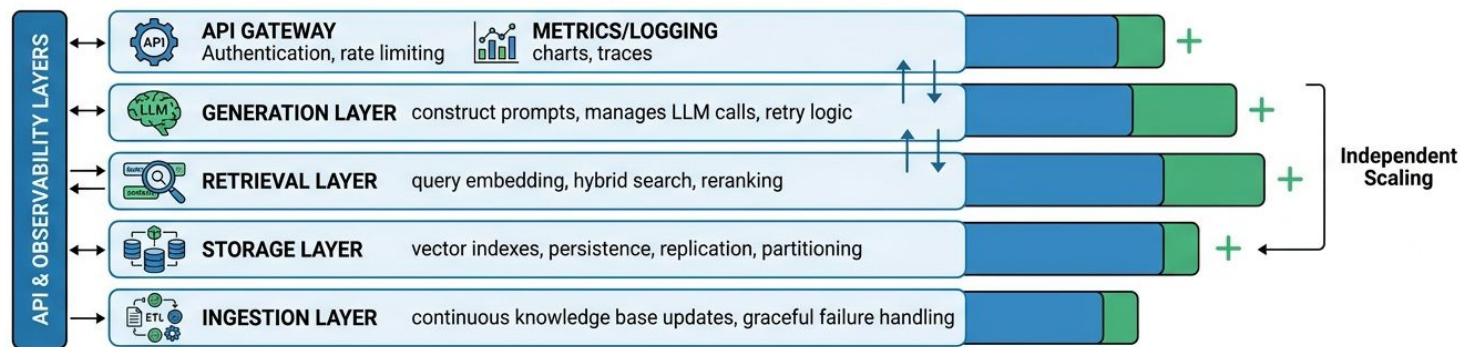
Chapter 6.5: Production RAG Architecture

US General Service Administration
AI Community of Practice - March 2026

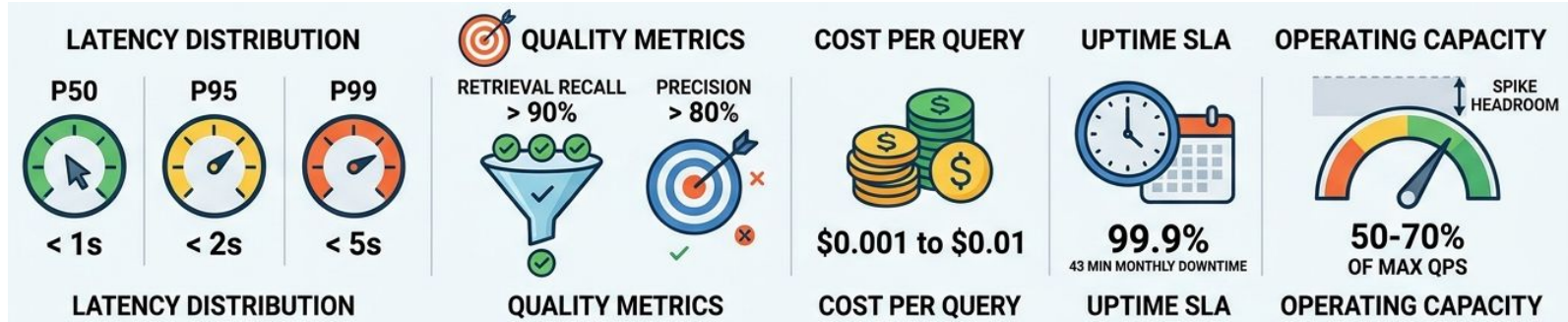
The Layered Architecture



The Layered RAG Architecture

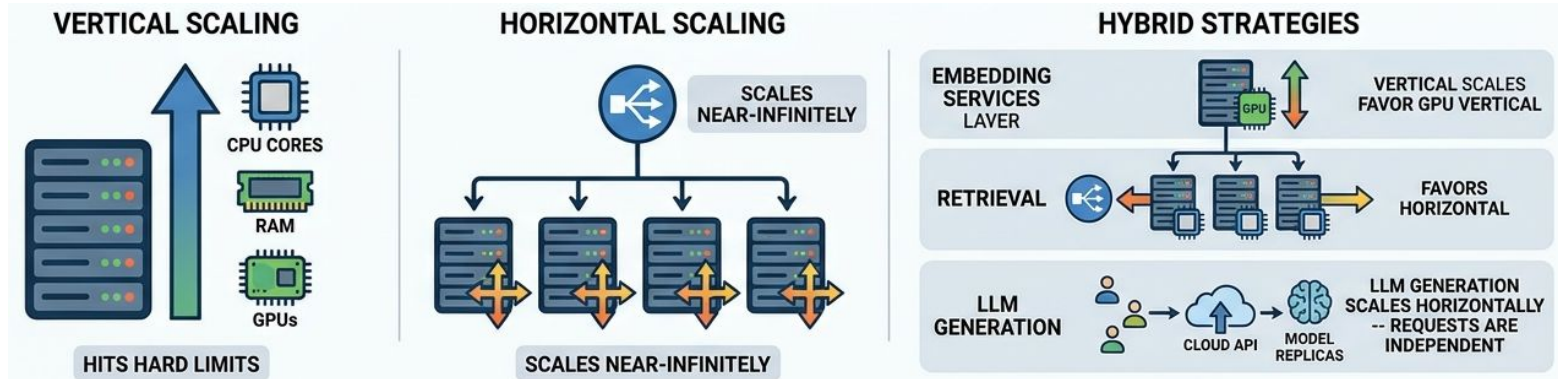


Success Metrics That Actually Matter



- Latency distribution: P50 under 1s, P95 under 2s, P99 under 5s
- Retrieval recall above 90%, precision above 80%
- Per-query cost typically \$0.001 to \$0.01
- Uptime SLA of 99.9% allows only 43 minutes monthly downtime
- Operate at 50-70% of max QPS for spike headroom

Scaling -- Vertical, Horizontal, and Hybrid



- Vertical scaling adds resources to existing servers; hits hard limits
- Horizontal scaling adds more instances; scales near-infininitely
- Hybrid strategies optimize each layer independently
- Embedding services favor GPU vertical; retrieval favors horizontal
- LLM generation scales horizontally -- requests are independent

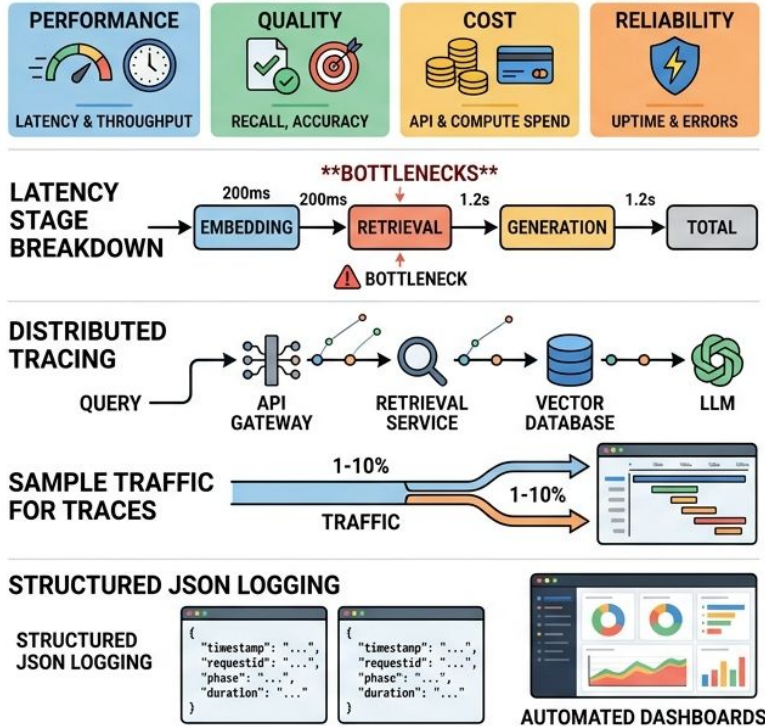
Caching -- The Highest-Impact Optimization

- Query embedding cache saves 50-100ms per repeated query
 - Semantic caching recognizes paraphrases at cosine similarity > 0.95
 - Retrieved context caching avoids repeated vector DB searches
 - Response caching returns answers in 10ms instead of 2 seconds
 - Hit rates of 40-60% are typical with semantic matching
- 

Cache Invalidation Strategies

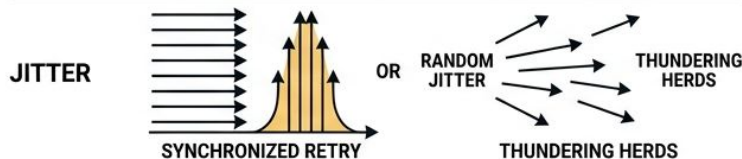
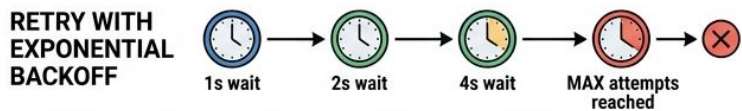
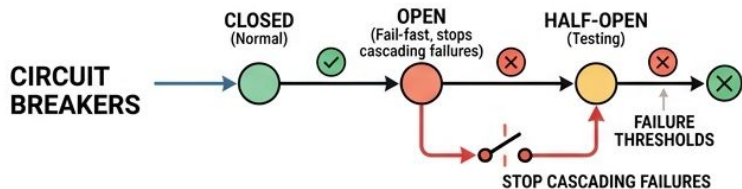
- Time-based (TTL): entries expire after configured duration
 - Event-based: invalidate when knowledge base documents change
 - LRU eviction prevents memory exhaustion under capacity limits
 - Combine all three for production-grade freshness guarantees
 - MD5 hash keys handle long queries within Redis limits
- 

Monitoring and Observability



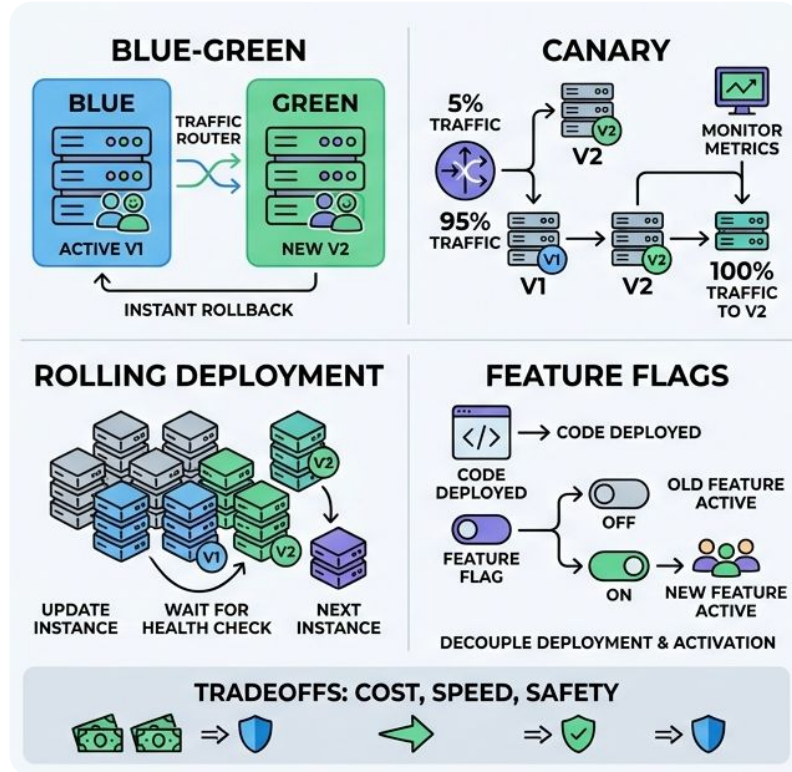
- Track four metric categories: performance, quality, cost, reliability
- Break latency into stages to identify specific bottlenecks
- Distributed tracing connects request flow across all services
- Sample 1-10% of traffic for traces to control overhead
- Structured JSON logging enables automated dashboards

Fault Tolerance and Graceful Degradation



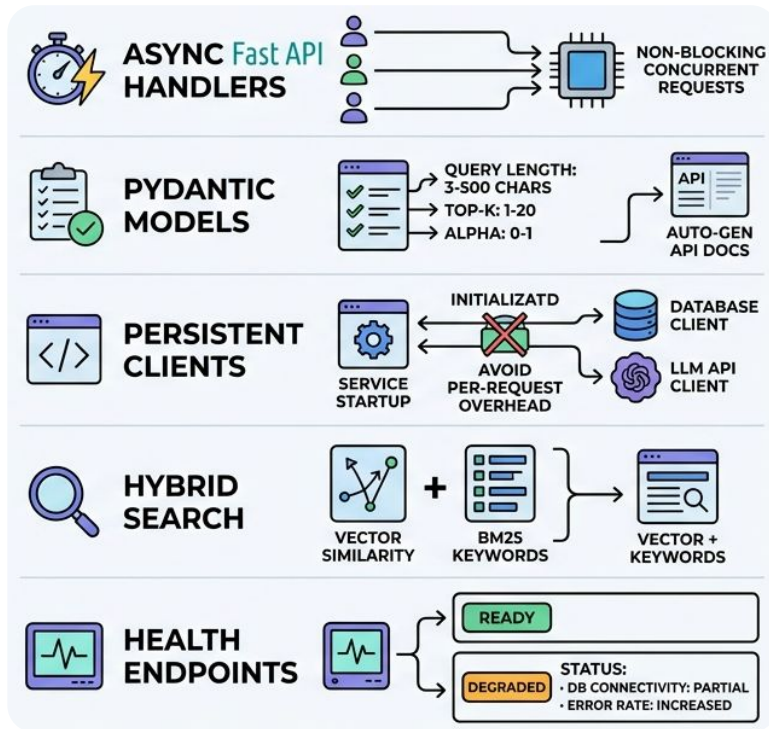
- Circuit breakers stop cascading failures when dependencies degrade
- Three states: closed (normal), open (fail-fast), half-open (testing)
- Graceful degradation: skip reranking, fall back to keyword search
- Retry with exponential backoff: 1s, 2s, 4s up to maximum
- Jitter prevents thundering herds from synchronized retries

Deployment Patterns for Zero Downtime



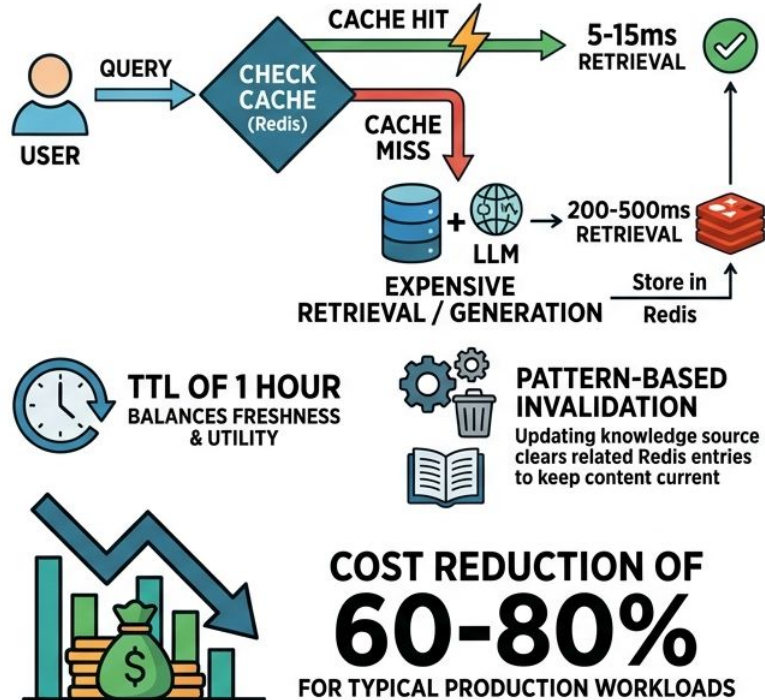
- Blue-green: two full environments; instant rollback via traffic switch
- Canary: route 5% to new version; expand if metrics hold
- Rolling deployment updates instances incrementally on same cluster
- Feature flags decouple code deployment from feature activation
- Each pattern trades cost, speed, and safety differently

Production RAG Service Implementation



- FastAPI with async handlers for non-blocking concurrent requests
- Pydantic models validate inputs and auto-generate API docs
- Persistent clients avoid per-request connection overhead
- Hybrid search combines vector similarity with BM25 keywords
- Health endpoints report readiness and degradation status

Caching in Practice with Redis



- Check cache before any expensive retrieval or generation
- Cache hits return in 5-15ms versus 200-500ms uncached
- TTL of 1 hour balances freshness with cache utility
- Pattern-based invalidation clears related entries on updates
- Cost reduction of 60-80% for typical production workloads

Key Takeaways

- Production RAG demands layered architecture with independent scaling
 - Caching alone cuts costs 60-80% and latency by orders of magnitude
 - Embrace failure: circuit breakers, retries, graceful degradation
 - Monitor everything: metrics, traces, alerts on SLO breaches
 - Next: Chapter 6.6 explores query decomposition for complex questions
- 